Procesos de Desarrollo de Software

CLASE 13 – PATRONES DE DISEÑO

Agenda de la clase

- Factory
- Facade

Factory

Es un patrón de **Creacional.**

¿Qué hace?

- Provee de una interface para crear familias de objetos sin especificar las clases concretas.

Se **sugiere** su utilización **cuando**:

- Cuando se necesita crear **familias de objetos relacionados** sin acoplarse a clases concretas.

Tipos de Fábricas

- **Factory Method:** Encapsula en un único punto (un **método**) la creación de objetos que tienen una interfaz común.
  - Una clase con if/switch crea objetos.
- **Simple Factory:** Encapsula en un único punto (**una clase**) la creación de objetos que tienen, al menos, una interfaz común.
  - Una superclase define el flujo y las subclases deciden qué objeto crear.
- **Abstract Factory:** Crea familias completas de objetos compatibles.

Beneficios

- Beneficios:
  - **Desacoplamiento:** el cliente no conoce las clases concretas.
  - Sin Factory:

```
Cafe cafe = new Cafe();
```

  - Con Factory:

```
Bebida bebida = factory.crearBebida();
```

  - Cumple **Open/Closed**: El cliente depende de la abstracción, no de clases concretas. Si mañana aparece CafePremium, CafeColombiano, etc. No hay que tocar el cliente. No modificamos consumidores.
  - **Centraliza la creación:** toda la lógica de construcción vive en un único lugar.
  - Oculta la complejidad de la creación.

```
new Cafe(
    proveedor,
    precio,
    ingredientes,
    descuento,
    impuestos,
    logger
);
```



```
factory.crearBebida();
```

Práctica

- Mejoremos el diseño planteado para el ejercicio de aplicación "**Módulo de Envío de Notificaciones**".
- Realicemos el ejercicio de aplicación de patrones "**Módulo exportador**"

Magic Café ☕ ✨

- Cómo sumar familias de menús y una entrada simple al sistema sin romper los patrones existentes.

Composite
Decorator
Strategy
State
Observer

+

Abstract Factory

Punto de partida

- El pedido ya podía armarse, decorarse, descontarse, avanzar por estados y notificar cambios.

Strategy	Decorator	Composite	State	Observer
Descuentos SinDescuento Estudiante Hechizo	**Extras** Leche Crema Canela	**Combos** Producto + Combo tratados igual	**Estados** Creado Pagado Preparando	**Avisos** Push Email Reporting

Ahora agregamos

Abstract Factory para crear familias completas de productos de Magic Café.

Problema: creación dispersa

- El problema no es vender un producto; el problema es crear conjuntos completos y consistentes.

```
Vendible cafe = new Producto("Café", 4500);
Vendible croissant = new Producto("Croissant", 3200);
Vendible muffin = new Producto("Muffin", 3600);

Combo combo = new Combo("Desayuno");
combo.agregarVendible(cafe, 1);
combo.agregarVendible(croissant, 2);
combo.agregarVendible(muffin, 1);
```

Problemas

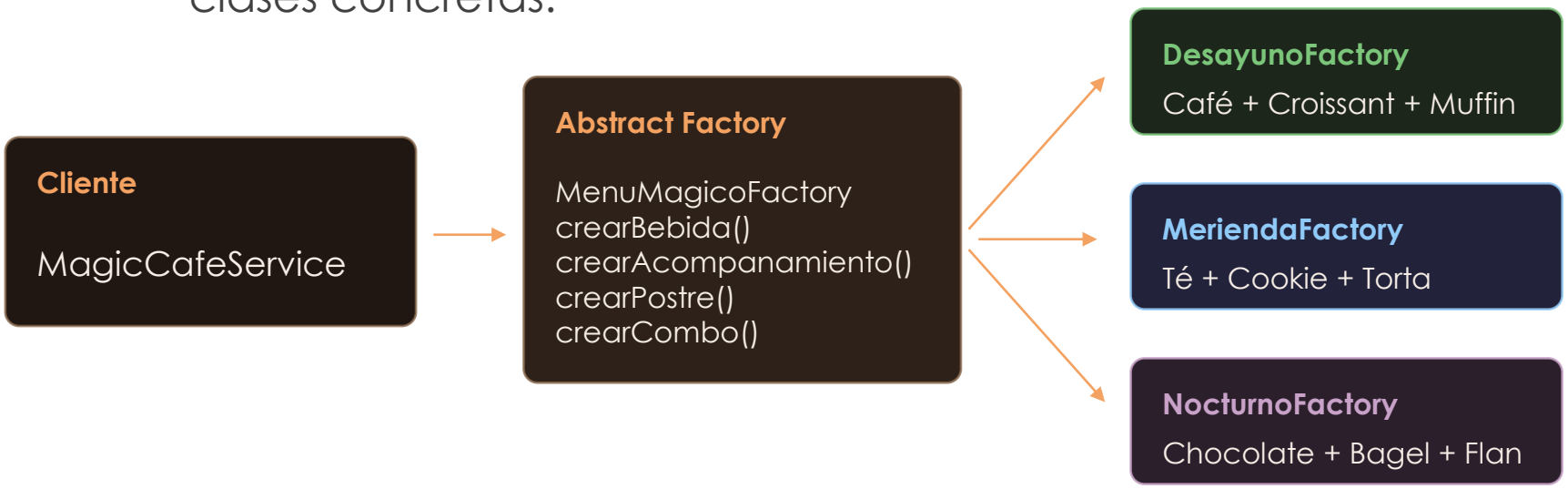
- El Service conoce demasiadas clases concretas.
- Si agrego un menú nuevo, crece la lógica de creación.
- Se puede mezclar por error una familia con otra.
- Baja el nivel de cohesión del caso de uso.

Idea

Sacar la creación de familias de objetos a factories especializadas.

Qué problema resuelve Abstract Factory

- Permite crear familias de objetos relacionados sin depender de clases concretas.



Idea clave

El cliente pide "dame una familia de menú". La factory concreta decide qué productos la componen.

Antes y después del Abstract Factory

- La mejora es mover la responsabilidad de creación fuera del caso de uso.

ANTES

MagicCafeService instancia productos y combos directamente.

new Producto("Café")
new Producto("Croissant")
new Combo("Desayuno")

Problema: el caso de uso se llena de detalles de construcción.

DESPUÉS

MagicCafeService recibe una abstracción:

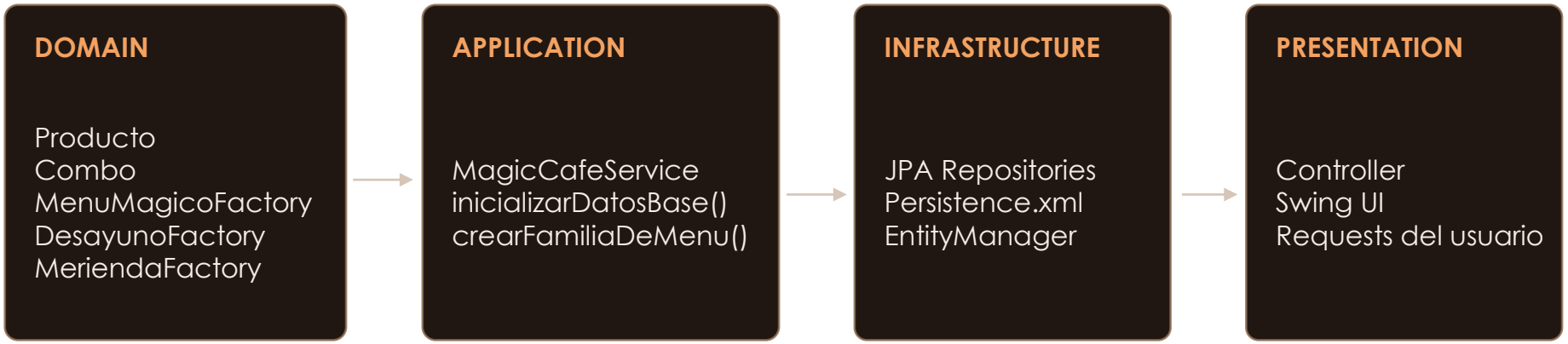
MenuMagicoFactory factory
factory.crearBebida()
factory.crearCombo(...)

Beneficio: agregar una familia nueva no rompe el flujo principal.

OCP + BAJO ACOPLAMIENTO

Piezas en Clean Architecture

- El dominio define el patrón; la aplicación decide cuándo usarlo.



Paso 1: crear el contrato de la familia

- La interfaz representa qué objetos debe saber crear una familia de menú.

MenuMagicoFactory

crearBebida()
crearAcompanamiento()
crearPostre()
crearCombo(...)

```
public interface MenuMagicoFactory {

    Producto crearBebida();
    Producto crearAcompanamiento();
    Producto crearPostre();

    Combo crearCombo(Vendible bebida,
                     Vendible acompanamiento,
                     Vendible postre);
}
```

Paso 2: agregar factories concretas

- Cada factory concreta sabe armar una familia compatible.

DesayunoMagicoFactory

Bebida: Café Latte
Acompañamiento: Croissant
Postre: Muffin
Combo: Desayuno Mágico

MeriendaMagicaFactory

Bebida: Té encantado
Acompañamiento: Cookie
Postre: Torta
Combo: Merienda Mágica

MenuNocturnoFactory

Bebida: Chocolate
Acompañamiento: Bagel
Postre: Flan
Combo: Nocturno Premium

```
MenuMagicoFactory factory = new DesayunoMagicoFactory();
Vendible bebida = factory.crearBebida();
Combo combo = factory.crearCombo(bebida, acompanamiento, postre);
```

Paso 3: Service usa la abstracción

- El caso de uso queda corto, legible y desacoplado de menús concretos.

```
private void crearFamiliaDeMenu(MenuMagicoFactory factory) {
    Vendible bebida = vendibleRepository.save(
        factory.crearBebida());
    Vendible acompanamiento = vendibleRepository.save(
        factory.crearAcompanamiento());
    Vendible postre = vendibleRepository.save(
        factory.crearPostre());

    Combo combo = factory.crearCombo(
        bebida, acompanamiento, postre);
    vendibleRepository.save(combo);
}
```

Bajo acoplamiento

El Service no sabe qué clase concreta representa cada producto.

Alta cohesión

Cada factory se encarga solamente de crear su familia.

Open/Closed

Nuevo menú = nueva factory, no cambio el flujo principal.

Flujo completo: creación de menús base

- El patrón aparece al inicializar o cargar familias de productos del catálogo.
- El catálogo se carga con familias completas sin que Presentation ni Infrastructure sepan cómo se componen.

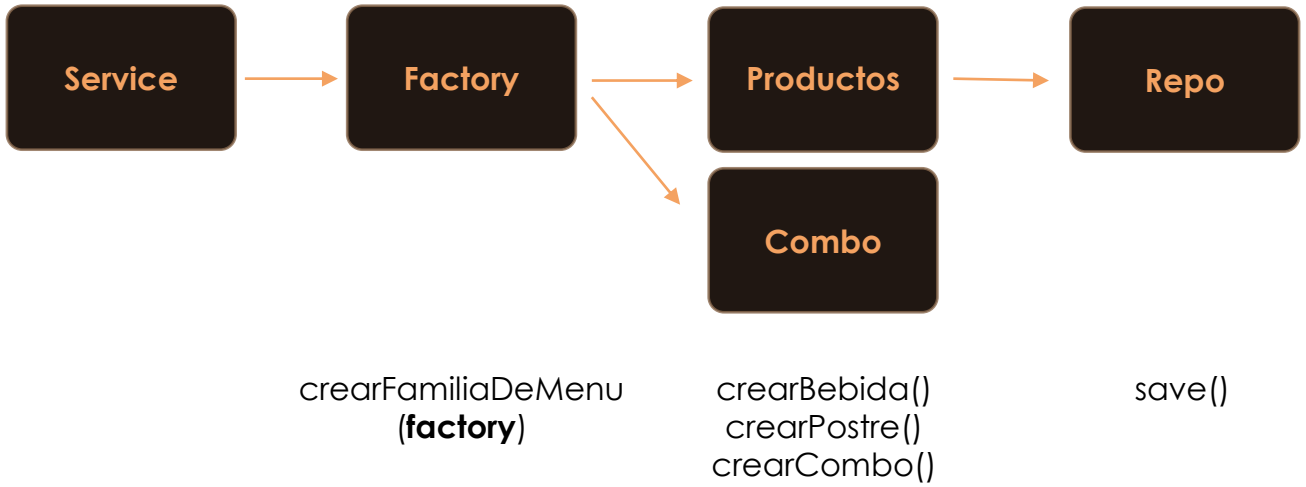
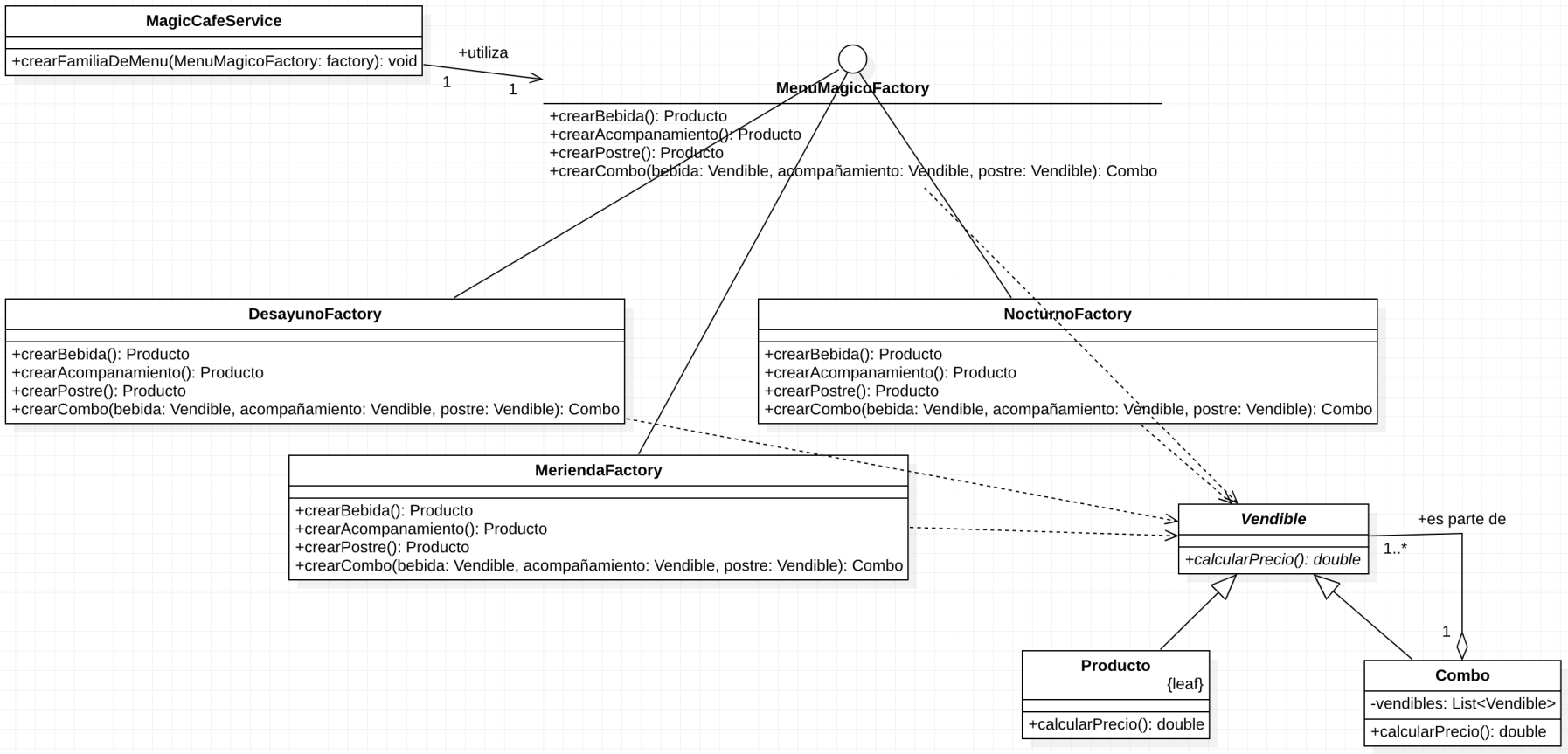


Diagrama de Clases (simplificado)



Por qué queda mejor diseñado

- Abstract Factory suma extensibilidad sin mezclar responsabilidades.

Bajo acoplamiento
GRASP

El Service depende de MenuMagicoFactory, no de cada producto concreto.

Alta cohesión
GRASP

Cada factory concentra la creación de una familia específica.

Open/Closed
SOLID

Se agrega un menú creando otra factory.

DIP
SOLID

El caso de uso trabaja con abstracciones.

```
// Nuevo menú sin tocar MagicCafeService
public class MenuNavidadFactory implements MenuMagicoFactory { ... }
```

Conclusión

- Cuando el sistema necesita crear una familia completa de productos, el Service usa una Abstract Factory. Así evita conocer clases concretas y garantiza que los objetos creados pertenecen al mismo menú.

1	2	3	4
El Service necesita cargar menús.	Elige una factory concreta.	La factory crea la familia.	El combo queda consistente.

Facade

Es un patrón de **Estructural.**

¿Qué hace?

- Expone una interfaz simple para la utilización de un componente de algún tercero, generalmente, considerado "complejo".
- Permite que dos o más soluciones/proyectos interactúen, manteniendo el acoplamiento al mínimo posible.

Se **sugiere** su utilización **cuando**:

- Se necesita exponer una interfaz sencilla para permitirle a un tercero la utilización de algún componente complejo, ocultado, de esta manera, la complejidad real.
- Se requiere hacer uso de un componente externo complejo, sin dejar fuertemente acoplada toda la solución con el mismo.

Componentes

- **Clase Facade**: Clase que reúne todos los componentes complejos/de terceros, y expone métodos "sencillos". Es la única clase que debería estar acoplada a los componentes complejos/de terceros y que debería brindar todas las funcionalidades necesarias.
- **Clases "Complejas" y/o "Externas"**: Clases que necesitan ser encapsuladas por "la fachada" ya que no interesa "quienes son", sino que interesa "lo que hacen".
- **Cliente**: Clase que hace uso de las funcionalidades provistas por los componentes de terceros a través de "la fachada".

Ejemplo

```
public class HomeTheaterFacade {

    private DVDPlayer dvd;
    private Amplifier amp;
    private Projector projector;

    public void watchMovie(String movie) {
        projector.on();
        amp.on();
        dvd.on();
        dvd.play(movie);
    }
}
```

- El cliente no necesita conocer todas las clases internas.
- Proporciona una **interfaz simplificada** para acceder a un subsistema complejo.

Controller: es un Facade?

```
@RestController
public class PedidoController {

    private PedidoService pedidoService;
    private NotificacionService notificacionService;

    @PostMapping("/pedidos")
    public void crearPedido() {
        pedidoService.crear();
        notificacionService.notificar();
    }
}
```

- Un **Controller** suele recibir una petición y coordinar varios objetos internos.
- Desde la perspectiva del cliente HTTP:
  - **POST** /pedidos
- El consumidor no conoce:
  - PedidoService
  - Repositorios
  - Notificadores
  - Transacciones
- Por eso muchos autores dicen que el Controller está cumpliendo un rol similar a una Facade.

Diferencias y Similitudes

¿Por qué se parecen?

- Ambos:
  - Ocultan complejidad
  - Exponen una interfaz más simple
  - Reducen acoplamiento
  - Actúan como punto de entrada

¿Por qué NO son exactamente lo mismo?

- Porque tienen responsabilidades distintas.

Facade	Controller
Patrón de diseño estructural	Recibe eventos externos
Simplifica un subsistema	Traduce solicitudes.
No está asociada a UI ni HTTP	Coordina casos de uso

Gracias!

- Preguntas?
- mailto: mstricagnoli@uade.edu.ar

